

Reviewer Pack — Minimal Symplectic Capacity of Real Algebraic Curves vs. Cir...

Entry 95a20cccef17 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 15:27:31 UTC

Minimal Symplectic Capacity of Real Algebraic Curves vs. Circuit Monotone Width

Entry ID: 95a20cccef17 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 15:27:31 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry (Real Algebraic Geometry) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every real algebraic curve C , the minimal symplectic capacity of its defining polynomial $f(x)$ is linearly correlated with the circuit monotone width of any circuit computing a symmetric function of the roots of $f(x)$, such that $\min_{\{g \in \text{Sym}(C)\}} \text{sym_cap}(f) = \Theta(\text{width_mon}(g))$, where $\text{sym_cap}(f)$ is the minimal symplectic capacity and $\text{width_mon}(g)$ is the circuit monotone width.

Rationale (proposer's reasoning):

The symmetry in the roots of a polynomial may expose underlying geometric structures, which could potentially lead to simpler circuits or reveal inherent complexity. Symplectic capacity measures the symmetry of real algebraic curves, while circuit monotone width provides a measure of circuit complexity. This conjecture aims to link these two concepts by suggesting that the symplectic capacity can be used as an indicator for circuit monotone width.

Taxonomy category: Symplectic_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1366ccc1b5bb35d7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of the seeds, the correlation coefficient between the computed symplectic capacity and circuit monotone width exceeds 0.7, with no seed having a symplectic capacity greater than 1.5 times its corresponding circuit monotone width.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symplectic geometry AND real algebraic geometry AND minimal symplectic capacity AND circuit monotone width - circuit monotone width IN symplectic geometry AND real algebraic curves - real algebraic curve AND defining polynomial AND minimal symplectic capacity related to circuit monotone width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A, b):
        n = len(b)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            factor = 1 / A[i][i]
            for j in range(n):
                A[i][j] *= factor
            b[i] *= factor
            for k in range(n):
                if k != i:
                    factor = A[k][i]
                    for j in range(n):
                        A[k][j] -= factor * A[i][j]
                    b[k] -= factor * b[i]
        return b

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
```

```

        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
    return C

def circuit_monotone_width(n):
    if n == 1:
        return 1
    width = 0
    for i in range(2, n + 1):
        width = max(width, circuit_monotone_width(i - 1) + 1)
    return width

def min_symplectic_capacity(n):
    # Simplified approximation for demonstration purposes
    return random.uniform(0.5, 1.5)

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_sym_cap = 0
total_width_mon = 0

for n in n_values:
    for _ in range(5):
        width_mon = circuit_monotone_width(n)
        sym_cap = min_symplectic_capacity(n)
        total_sym_cap += sym_cap
        total_width_mon += width_mon
        instances_tested += 1

mean_sym_cap = total_sym_cap / instances_tested
mean_width_mon = total_width_mon / instances_tested
correlation_coefficient = (instances_tested * sum(s * w for s, w in zip(total_sym_cap, total_width_mon)) -
                           total_sym_cap * total_width_mon) / math.sqrt(
    instances_tested * sum(s ** 2 for s in total_sym_cap) - total_sym_cap ** 2 *
    instances_tested * sum(w ** 2 for w in total_width_mon) - total_width_mon ** 2)

conjecture_holds = correlation_coefficient > 0.7 and all(1.5 * w >= c for c, w in zip(total_sym_cap, total_width_mon))
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\\"seed\\"}: {seed}, **{{result}}}")

```

```

results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.95:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete its intended calculations and verify the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16378
2	preregistration	ollama_remote	glm4:latest	0	0	20229
3	novelty	ollama_remote	glm4:latest	0	0	8792
4	novelty	ollama_remote	glm4:latest	0	0	23912
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33549
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28695
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14401
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17079
9	judge	ollama_remote	glm4:latest	0	0	8400

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 171436 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/95a20cccef17.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/95a20cccef17.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/95a20cccef17.tar.gz` (if generated)